

Part 4

Conditional Statements

Topics Covered

- if Statement
- if...else Statement
- if...elif...else Statement
- Nested if
- Short-Hand if
- Short-Hand if...else
- Using Logical Operators with Conditions

Learning Objectives

By the end of this lesson, you will be able to:

- Understand conditional statements.
- Write programs using `if`.
- Use `if...else` for decision making.
- Work with multiple conditions using `elif`.
- Write nested `if` statements.
- Use short-hand conditional statements.
- Combine conditions using logical operators.
- Develop simple decision-making programs.

What are Conditional Statements?

Conditional statements allow a program to make decisions based on whether a condition is **True** or **False**.

They help control the flow of a program by executing different blocks of code depending on the result of a condition.

Example

```
In [ ]: age = 18

if age >= 18:
    print("Eligible to vote")
```

If the condition is **True**, the code inside the `if` block is executed.

Why Do We Need Conditional Statements?

Conditional statements help programs make decisions.

They are used to:

- Check conditions.
- Execute different code based on different situations.
- Validate user input.
- Build interactive programs.
- Control program flow.

Without conditional statements, every program would execute the same instructions every time.

Types of Conditional Statements

Python provides the following conditional statements:

Statement	Purpose
<code>if</code>	Executes code if a condition is true
<code>if...else</code>	Chooses between two blocks
<code>if...elif...else</code>	Chooses from multiple conditions
Nested <code>if</code>	Places one <code>if</code> inside another
Short-Hand <code>if</code>	One-line <code>if</code> statement
Short-Hand <code>if...else</code>	One-line <code>if...else</code> statement

`if` Statement

The `if` statement is the simplest conditional statement in Python. It is used to execute a block of code **only if a specified condition is True**.

If the condition is `False`, the code inside the `if` block is skipped.

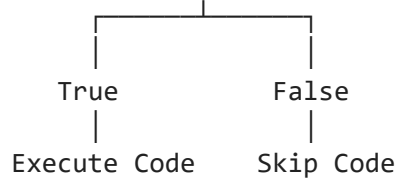
Syntax

```
if condition:  
    # Code to execute if the condition is True
```

Note: The code inside the `if` block must be **indented**.

Flow Diagram

Condition
|



Example 1

age = 20

```
if age >= 18:  
    print("Eligible to vote")
```

Output

Eligible to vote

Explanation

Since `20 >= 18` is **True**, the code inside the `if` block is executed.

Example 2

marks = 85

```
if marks >= 50:  
    print("Pass")
```

Output

Pass

Explanation

Since `85 >= 50` is **True**, the message "**Pass**" is displayed.

Example 3

temperature = 35

```
if temperature > 30:  
    print("It is a hot day.")
```

Output

It is a hot day.

Key Points

- The `if` statement checks whether a condition is **True**.
- The code inside the `if` block executes only when the condition is **True**.
- If the condition is **False**, the `if` block is skipped.
- A colon `:` must be placed after the condition.
- Proper indentation is required inside the `if` block.

if...else Statement

The **if...else statement** is used to execute one block of code if a condition is **True** and another block of code if the condition is **False**.

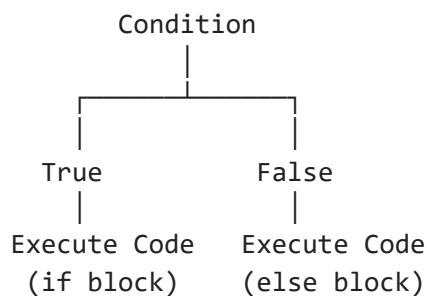
It allows a program to choose between **two possible outcomes**.

Syntax

```
if condition:
    # Code to execute if the condition is True
else:
    # Code to execute if the condition is False
```

Note: Both the **if** and **else** blocks must be properly indented.

Flow Diagram



Example 1

```
age = 16
```

```
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

Output

Not eligible to vote

Explanation

Since `16 >= 18` is **False**, the code inside the **else** block is executed.

Example 2

```
number = 8
```

```
if number % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output

Even Number

Explanation

Since `8 % 2 == 0` is **True**, the program prints "**Even Number**".

Example 3

```
marks = 45
```

```
if marks >= 50:
    print("Pass")
else:
    print("Fail")
```

Output

Fail

Explanation

Since `45 >= 50` is **False**, the program prints "**Fail**".

Key Points

- The `if...else` statement is used when there are **two possible outcomes**.
- If the condition is `True`, the `if` block executes.
- If the condition is `False`, the `else` block executes.
- Only **one** of the two blocks is executed.
- A colon `:` is required after both `if` and `else`.
- Proper indentation is required for both blocks.

`if...elif...else` Statement

The `if...elif...else` statement is used when there are **multiple conditions** to check. Python evaluates each condition from top to bottom and executes the block of the **first condition that is True**.

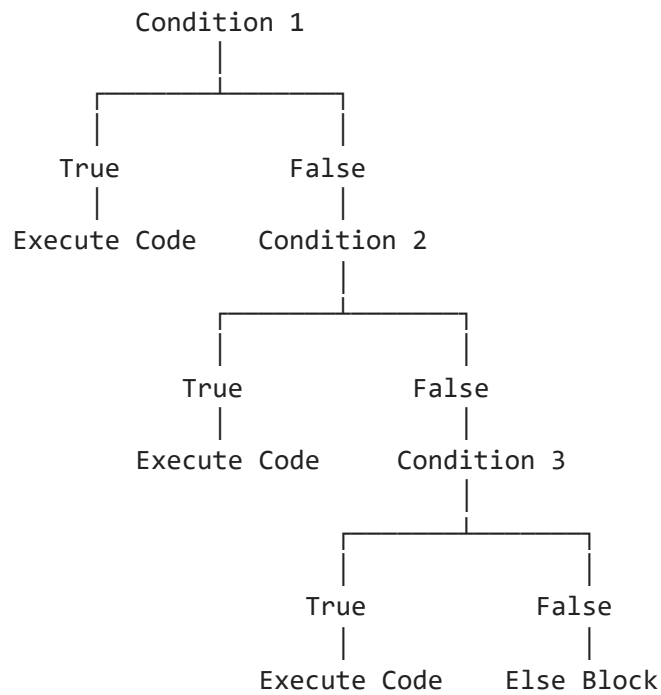
If none of the conditions are `True`, the `else` block is executed.

Syntax

```
if condition1:
    # Code to execute if condition1 is True
elif condition2:
    # Code to execute if condition2 is True
elif condition3:
    # Code to execute if condition3 is True
else:
    # Code to execute if all conditions are False
```

Note: You can use one or more `elif` statements between `if` and `else`.

Flow Diagram



Example 1

marks = 82

```
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output

Grade B

Explanation

Since `82 >= 75` is **True**, the program prints "**Grade B**" and skips the remaining conditions.

Example 2

number = 0

```
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Output

Zero

Explanation

The number is neither positive nor negative, so the `else` block is executed.

Example 3

```
day = 6
```

```
if day == 1:
    print("Monday")
elif day == 2:
    print("Tuesday")
elif day == 3:
    print("Wednesday")
else:
    print("Another Day")
```

Output

Another Day

Key Points

- Use `if...elif...else` to check multiple conditions.
- Python checks conditions from top to bottom.
- Only the **first True** condition is executed.
- The remaining conditions are skipped.
- The `else` block runs only if all conditions are `False`.
- Proper indentation is required for all blocks.

Nested if Statement

A **Nested if Statement** is an `if` statement placed **inside another if statement**.

The inner `if` statement is executed only if the outer `if` condition is `True`.

Nested `if` statements are useful when a decision depends on multiple related conditions.

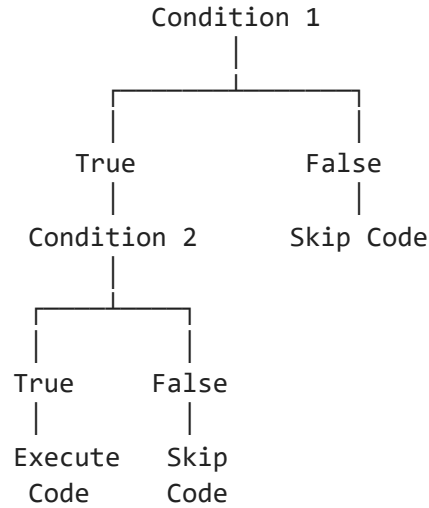
Syntax

```
if condition1:
    # Code

    if condition2:
        # Code
```

Note: The inner `if` statement must be properly indented inside the outer `if` block.

Flow Diagram



Example 1

```
age = 20  
has_id = True
```

```
if age >= 18:  
    if has_id:  
        print("Entry Allowed")
```

Output

Entry Allowed

Explanation

- The outer condition `age >= 18` is `True`.
- The inner condition `has_id` is also `True`.
- Therefore, **"Entry Allowed"** is displayed.

Example 2

```
number = 15
```

```
if number > 0:  
    if number % 2 == 0:  
        print("Positive Even Number")
```

Output

Explanation

The outer condition is `True`, but the inner condition is `False`. Therefore, no output is displayed.

Example 3

```
username = "admin"  
password = "python123"
```

```
if username == "admin":  
    if password == "python123":  
        print("Login Successful")
```


Output

Login Successful

Key Points

- A nested `if` is an `if` statement inside another `if`.
- The inner `if` is checked only if the outer `if` condition is `True`.
- Nested `if` statements are useful when multiple conditions depend on each other.
- Proper indentation is required for both the outer and inner `if` blocks.
- Avoid excessive nesting, as it can make code harder to read.

Short-Hand `if`

The **Short-Hand `if`** statement allows you to write a simple `if` statement in **one line**. It is useful when there is only **one statement** to execute if the condition is `True`.

Syntax

```
if condition: statement
```

Note: Short-Hand `if` should only be used for simple conditions. For multiple statements, use the standard `if` statement.

Example 1

```
age = 20
```

```
if age >= 18: print("Eligible to vote")
```

Output

Eligible to vote

Explanation

Since `20 >= 18` is `True`, the statement is executed.

Example 2

```
number = 8
```

```
if number % 2 == 0: print("Even Number")
```

Output

Even Number

Explanation

Since `8 % 2 == 0` is `True`, the program prints **"Even Number"**.

Example 3

```
marks = 45
```

```
if marks >= 50: print("Pass")
```

Output

Explanation

Since `45 >= 50` is **False**, nothing is printed.

Key Points

- Short-Hand `if` is a one-line version of the `if` statement.
- It is suitable when only one statement needs to be executed.
- If the condition is `False`, the statement is skipped.
- Avoid using Short-Hand `if` for complex logic or multiple statements.

Short-Hand `if...else` (Ternary Operator)

The **Short-Hand** `if...else`, also known as the **Ternary Operator**, is a one-line way to write an `if...else` statement.

It is useful when you want to choose between **two values or actions** in a single line of code.

Syntax

```
statement_if_true if condition else statement_if_false
```

Note: Use the ternary operator only for simple conditions. For complex logic, use the standard `if...else` statement.

Example 1

```
age = 20
```

```
print("Eligible") if age >= 18 else print("Not Eligible")
```

Output

```
Eligible
```

Explanation

Since `20 >= 18` is **True**, `"Eligible"` is printed.

Example 2

```
number = 7
```

```
print("Even") if number % 2 == 0 else print("Odd")
```

Output

Odd

Explanation

Since `7 % 2 == 0` is **False**, `"Odd"` is printed.

Example 3

```
a = 15
```

```
b = 20
```

```
largest = a if a > b else b
```

```
print(largest)
```

Output

20

Explanation

Since `15 > 20` is **False**, the value of `b` is assigned to `largest`.

Key Points

- The ternary operator is a one-line version of the `if...else` statement.
- It is best suited for simple conditions.
- If the condition is `True`, the first expression is executed.
- If the condition is `False`, the second expression is executed.
- Avoid using the ternary operator for complex or lengthy conditions.

Using `and`, `or`, `not` with Conditions

Logical operators allow you to combine or modify conditions in conditional statements.

Python provides three logical operators:

- `and`
- `or`
- `not`

These operators are commonly used with `if` statements to make more complex decisions.

Using `and`

The `and` operator returns `True` only if **both conditions** are `True`.

Syntax

```
if condition1 and condition2:  
    # Code
```

Example

```
age = 20  
has_id = True
```

```
if age >= 18 and has_id:  
    print("Entry Allowed")
```

Output

Entry Allowed

Explanation

Both conditions are `True` , so the `if` block is executed.

Using or

The `or` operator returns `True` if **at least one condition** is `True` .

Syntax

```
if condition1 or condition2:  
    # Code
```

Example

```
day = "Sunday"
```

```
if day == "Saturday" or day == "Sunday":  
    print("Weekend")
```

Output

Weekend

Explanation

One of the conditions is `True` , so the program prints **"Weekend"**.

Using not

The `not` operator reverses the result of a condition.

- `True` becomes `False`
- `False` becomes `True`

Syntax

```
if not condition:  
    # Code
```

Example

```
is_logged_in = False
```

```
if not is_logged_in:  
    print("Please Log In")
```

Output

Please Log In

Explanation

Since `is_logged_in` is `False`, `not False` becomes `True`, so the message is displayed.

Example Using Multiple Conditions

```
age = 22
```

```
citizen = True
```

```
if age >= 18 and citizen:  
    print("Eligible to Vote")  
else:  
    print("Not Eligible")
```

Output

Eligible to Vote

Key Points

- Use `and` when **all conditions** must be `True`.
- Use `or` when **at least one condition** must be `True`.
- Use `not` to reverse the result of a condition.
- Logical operators are commonly used with `if`, `if...else`, and `if...elif...else` statements.
- Combine conditions carefully to make programs more efficient and readable.

Common Errors

- Forgetting the colon (`:`) after `if`, `elif`, or `else`.
- Incorrect indentation inside conditional blocks.
- Using `=` instead of `==` for comparison.
- Writing conditions in the wrong order.
- Forgetting to handle all possible cases with `else`.

Best Practices

- Use meaningful and clear conditions.
- Keep conditional statements simple.
- Use proper indentation.
- Use `elif` instead of multiple `if` statements when checking related conditions.
- Add comments for complex conditions if needed.
- Test your program with different input values.

Simple Programs (10)

The following programs demonstrate the use of conditional statements.

Program 1

Write a Python program to check whether a number is positive.

Program 2

Write a Python program to check whether a person is eligible to vote.

Program 3

Write a Python program to check whether a number is even or odd.

Program 4

Write a Python program to find the larger of two numbers.

Program 5

Write a Python program to display a grade based on marks using `if...elif...else`.

Program 6

Write a Python program to check whether a year is a leap year.

Program 7

Write a Python program to display the day of the week based on a number (1–7) using `if...elif...else`.

Program 8

Write a Python program to check whether a character is a vowel or a consonant.

Program 9

Write a Python program to check whether a number lies between 1 and 100 using the `and` operator.

Program 10

Write a Python program to find the largest of three numbers using nested `if` statements.

Practice Programs (10)

Solve the following programs on your own before checking the solutions.

Program 1

Write a Python program to check whether a number is negative, positive, or zero.

Program 2

Write a Python program to check whether a person is eligible to apply for a driving license (minimum age: 18).

Program 3

Write a Python program to find the smaller of two numbers.

Program 4

Write a Python program to check whether a student has passed or failed (passing marks: 40).

Program 5

Write a Python program to classify a person's age as:

- Child
 - Teenager
 - Adult
 - Senior Citizen
-

Program 6

Write a Python program to check whether a number is divisible by both 3 and 5.

Program 7

Write a Python program to check whether a character is an uppercase or lowercase letter.

Program 8

Write a Python program to check whether a number is a multiple of 7 or 11.

Program 9

Write a Python program to display the season based on the month number (1–12).

Program 10

Write a Python program to determine whether a triangle is valid based on the sum of its three angles.

Cheat Sheet

Conditional Statements

Statement	Purpose
if	Executes code if the condition is True
if...else	Chooses between two blocks of code
if...elif...else	Checks multiple conditions
Nested if	Uses an if statement inside another if
Short-Hand if	One-line if statement
Short-Hand if...else	One-line if...else statement

Comparison Operators

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical Operators

Operator	Description
<code>and</code>	Returns <code>True</code> if both conditions are <code>True</code>
<code>or</code>	Returns <code>True</code> if at least one condition is <code>True</code>
<code>not</code>	Reverses the result of a condition

Basic Syntax

`if`

```
if condition:
    statement
```

`if...else`

```
if condition:
    statement
else:
    statement
```

`if...elif...else`

```
if condition:
    statement
elif condition:
    statement
else:
    statement
```

Nested `if`

```
if condition1:
    if condition2:
        statement
```

Short-Hand `if`

```
if condition: statement
```

Short-Hand `if...else`

```
statement1 if condition else statement2
```

Quick Tips

- Use `if` for one condition.
- Use `if...else` for two possible outcomes.
- Use `if...elif...else` for multiple conditions.
- Use `and` when all conditions must be `True`.
- Use `or` when at least one condition must be `True`.

- Use `not` to reverse a condition.
- Always use proper indentation.

Summary

In this lesson, you learned about **Conditional Statements**, which allow Python programs to make decisions based on conditions.

In this lesson, you learned:

- What conditional statements are.
 - Why conditional statements are important.
 - Different types of conditional statements:
 - `if`
 - `if...else`
 - `if...elif...else`
 - Nested `if`
 - Short-Hand `if`
 - Short-Hand `if...else`
 - How to combine conditions using:
 - `and`
 - `or`
 - `not`
 - Common errors and best practices.
 - How to write simple decision-making programs.
-

Key Takeaways

- Conditional statements control the flow of a program.
 - Conditions evaluate to either `True` or `False`.
 - Use `if` for a single condition.
 - Use `if...else` for two possible outcomes.
 - Use `if...elif...else` for multiple conditions.
 - Use logical operators to combine conditions.
 - Write clean, readable, and properly indented code.
-

What's Next?

In **Part 5**, you will learn about **Loops**, including:

- `for` Loop
- `while` Loop
- `break` Statement
- `continue` Statement
- `pass` Statement
- Nested Loops

Loops help you execute a block of code multiple times without writing the same code repeatedly.

